

PL/SQL Exercises

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag `IsVIP` to `TRUE` for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
1.
2. BEGIN
3.     EXECUTE IMMEDIATE 'DROP TABLE loans';
4. EXCEPTION WHEN OTHERS THEN NULL;
5. END;
6. /
7. BEGIN
8.     EXECUTE IMMEDIATE 'DROP TABLE customers';
9. EXCEPTION WHEN OTHERS THEN NULL;
10. END;
11. /
12.
13. CREATE TABLE customers (
14.     customer_id    NUMBER PRIMARY KEY,
15.     customer_name  VARCHAR2(100),
16.     age            NUMBER,
17.     balance        NUMBER,
18.     is_vip        VARCHAR2(5) DEFAULT 'FALSE'
19. );
20.
21. CREATE TABLE loans (
22.     loan_id        NUMBER PRIMARY KEY,
23.     customer_id    NUMBER REFERENCES customers(customer_id),
24.     loan_amount    NUMBER,
25.     interest_rate  NUMBER,
```

```

26.     due_date      DATE
27.);
28.
29.INSERT INTO customers VALUES (1, 'Ravi Kumar', 65, 15000, 'FALSE');
30.INSERT INTO customers VALUES (2, 'Anita Sharma', 45, 12000, 'FALSE');
31.INSERT INTO customers VALUES (3, 'Suresh Rao', 70, 8000, 'FALSE');
32.INSERT INTO customers VALUES (4, 'Priya Singh', 30, 5000, 'FALSE');
33.
34.INSERT INTO loans VALUES (101, 1, 200000, 8.5, SYSDATE + 10);
35.INSERT INTO loans VALUES (102, 2, 150000, 9.0, SYSDATE + 45);
36.INSERT INTO loans VALUES (103, 3, 100000, 7.5, SYSDATE + 20);
37.INSERT INTO loans VALUES (104, 4, 50000, 10.0, SYSDATE + 60);
38.
39.COMMIT;
40.
41.-- ----- THE ACTUAL LOGIC -----
42.SET SERVEROUTPUT ON;
43.
44.BEGIN
45.    FOR cust IN (SELECT customer_id, customer_name, age FROM customers)
46.    LOOP
47.        IF cust.age > 60 THEN
48.            UPDATE loans
49.            SET interest_rate = interest_rate - 1
50.            WHERE customer_id = cust.customer_id;
51.            DBMS_OUTPUT.PUT_LINE('Discount applied for ' ||
cust.customer_name ||
52.                                ' (Age: ' || cust.age || ')');
53.        END IF;
54.    END LOOP;
55.    COMMIT;
56.END;
57./
58.
59.-- Verify
60.SELECT * FROM loans ORDER BY loan_id;

```

Output:

Discount applied for Ravi Kumar (Age: 65)

Discount applied for Suresh Rao (Age: 70)

LOAN_ID	CUSTOMER_ID	LOAN_AMOUNT	INTEREST_RATE	DUE_DATE
---------	-------------	-------------	---------------	----------

101	1	200000	7.5	01-JUL-26
102	2	150000	9	05-AUG-26
103	3	100000	6.5	11-JUL-26
104	4	50000	10	20-AUG-26

LOAN_ID	CUSTOMER_ID	LOAN_AMOUNT	INTEREST_RATE	DUE_DATE
---------	-------------	-------------	---------------	----------

101	1	200000	7.5	01-JUL-26
102	2	150000	9	05-AUG-26
103	3	100000	6.5	11-JUL-26
104	4	50000	10	20-AUG-26

BEGIN

EXECUTE IMMEDIATE 'DROP TABLE customers';

EXCEPTION WHEN OTHERS THEN NULL;

END;

/

CREATE TABLE customers (

customer_id NUMBER PRIMARY KEY,

customer_name VARCHAR2(100),

age NUMBER,

balance NUMBER,

is_vip VARCHAR2(5) DEFAULT 'FALSE'

);

```
INSERT INTO customers VALUES (1, 'Ravi Kumar', 65, 15000, 'FALSE');
INSERT INTO customers VALUES (2, 'Anita Sharma', 45, 12000, 'FALSE');
INSERT INTO customers VALUES (3, 'Suresh Rao', 70, 8000, 'FALSE');
INSERT INTO customers VALUES (4, 'Priya Singh', 30, 5000, 'FALSE');
```

```
COMMIT;
```

```
-- ---- THE ACTUAL LOGIC ----
```

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
    FOR cust IN (SELECT customer_id, customer_name, balance FROM customers)
LOOP
```

```
    IF cust.balance > 10000 THEN
```

```
        UPDATE customers
```

```
        SET is_vip = 'TRUE'
```

```
        WHERE customer_id = cust.customer_id;
```

```
        DBMS_OUTPUT.PUT_LINE(cust.customer_name ||
```

```
                               ' promoted to VIP (Balance: ' || cust.balance || ')');
```

```
    END IF;
```

```
END LOOP;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
-- Verify
```

```
SELECT * FROM customers ORDER BY customer_id;
```

Output:

Ravi Kumar promoted to VIP (Balance: 15000)
Anita Sharma promoted to VIP (Balance: 12000)

CUSTOMER_ID

CUSTOMER_NAME

AGE BALANCE IS_VI

1

Ravi Kumar

65 15000 TRUE

2

Anita Sharma

45 12000 TRUE

CUSTOMER_ID

CUSTOMER_NAME

AGE BALANCE IS_VI

3

Suresh Rao

70 8000 FALSE

4

BEGIN

EXECUTE IMMEDIATE 'DROP TABLE loans';

EXCEPTION WHEN OTHERS THEN NULL;

END;

/

BEGIN

EXECUTE IMMEDIATE 'DROP TABLE customers';

EXCEPTION WHEN OTHERS THEN NULL;

END;

/

```
CREATE TABLE customers (  
    customer_id NUMBER PRIMARY KEY,  
    customer_name VARCHAR2(100),  
    age NUMBER,  
    balance NUMBER,  
    is_vip VARCHAR2(5) DEFAULT 'FALSE'  
);
```

```
CREATE TABLE loans (  
    loan_id NUMBER PRIMARY KEY,  
    customer_id NUMBER REFERENCES customers(customer_id),  
    loan_amount NUMBER,  
    interest_rate NUMBER,  
    due_date DATE  
);
```

```
INSERT INTO customers VALUES (1, 'Ravi Kumar', 65, 15000, 'FALSE');
```

```
INSERT INTO customers VALUES (2, 'Anita Sharma', 45, 12000, 'FALSE');
```

```
INSERT INTO customers VALUES (3, 'Suresh Rao', 70, 8000, 'FALSE');
```

```
INSERT INTO customers VALUES (4, 'Priya Singh', 30, 5000, 'FALSE');
```

```
INSERT INTO loans VALUES (101, 1, 200000, 8.5, SYSDATE + 10);
```

```
INSERT INTO loans VALUES (102, 2, 150000, 9.0, SYSDATE + 45);
```

```
INSERT INTO loans VALUES (103, 3, 100000, 7.5, SYSDATE + 20);
```

```
INSERT INTO loans VALUES (104, 4, 50000, 10.0, SYSDATE + 60);
```

```

COMMIT;

-- ---- THE ACTUAL LOGIC ----

SET SERVEROUTPUT ON;

DECLARE

    CURSOR due_loans_cur IS

        SELECT l.loan_id, l.due_date, c.customer_name

        FROM loans l

        JOIN customers c ON c.customer_id = l.customer_id

        WHERE l.due_date BETWEEN SYSDATE AND SYSDATE + 30;

BEGIN

    FOR rec IN due_loans_cur LOOP

        DBMS_OUTPUT.PUT_LINE('Reminder: Dear ' || rec.customer_name ||

                               ' ; your loan (ID: ' || rec.loan_id ||

                               ') is due on ' || TO_CHAR(rec.due_date, 'DD-MON-YYYY'));

    END LOOP;

END;

/

```

Output:

```

Reminder: Dear Ravi Kumar, your loan (ID: 101) is due on 01-JUL-2026
Reminder: Dear Suresh Rao, your loan (ID: 103) is due on 11-JUL-2026

```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
1.
2.  BEGIN
3.      EXECUTE IMMEDIATE 'DROP TABLE savings_accounts';
4.  EXCEPTION WHEN OTHERS THEN NULL;
5.  END;
6.  /
7.
8.  CREATE TABLE savings_accounts (
9.      account_id    NUMBER PRIMARY KEY,
10.     customer_name  VARCHAR2(100),
11.     account_type   VARCHAR2(20),
12.     balance        NUMBER(12,2)
13. );
14.
15. INSERT INTO savings_accounts VALUES (301, 'Ravi Kumar', 'SAVINGS',
16.    50000);
17. INSERT INTO savings_accounts VALUES (302, 'Anita Sharma', 'SAVINGS',
18.    25000);
19. INSERT INTO savings_accounts VALUES (303, 'Suresh Rao', 'CURRENT',
20.    10000);
21. INSERT INTO savings_accounts VALUES (304, 'Priya Singh', 'SAVINGS',
22.    8000);
23.
```



```

20. COMMIT;
21.
22. -- ----- THE PROCEDURE -----
23. CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
24.     v_interest_rate CONSTANT NUMBER := 0.01; -- 1%
25. BEGIN
26.     FOR acc IN (
27.         SELECT account_id, customer_name, balance
28.         FROM savings_accounts
29.         WHERE account_type = 'SAVINGS'
30.     ) LOOP
31.         UPDATE savings_accounts
32.         SET balance = balance + (balance * v_interest_rate)
33.         WHERE account_id = acc.account_id;
34.
35.         DBMS_OUTPUT.PUT_LINE('Interest applied to ' ||
acc.customer_name ||
36.                                ' (Account: ' || acc.account_id ||
37.                                ') - Old balance: ' || acc.balance ||
38.                                ', New balance: ' || ROUND(acc.balance *
(1 + v_interest_rate), 2));
39.     END LOOP;
40.     COMMIT;
41. EXCEPTION
42.     WHEN OTHERS THEN
43.         ROLLBACK;
44.         DBMS_OUTPUT.PUT_LINE('Error processing interest: ' ||
SQLERRM);
45.         RAISE;
46. END;
47. /
48.
49. -- ----- TEST CALL -----
50. SET SERVEROUTPUT ON;
51.
52. EXEC ProcessMonthlyInterest;
53.
54. -- Verify
55. SELECT * FROM savings_accounts ORDER BY account_id;

```

Output:

Interest applied to Ravi Kumar (Account: 301) – Old balance: 50000, New balance: 50500

Interest applied to Anita Sharma (Account: 302) – Old balance: 25000, New balance: 25250

Interest applied to Priya Singh (Account: 304) – Old balance: 8000, New balance: 8080

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE BALANCE

301

Ravi Kumar

SAVINGS 50500

302

Anita Sharma

SAVINGS 25250

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE BALANCE

303

Suresh Rao

CURRENT 10000

304

Priya Singh

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE BALANCE

SAVINGS 8080

Output:

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE

BALANCE

301

Ravi Kumar

SAVINGS

50500

302

Anita Sharma

SAVINGS

25250

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE

BALANCE

303

303

Suresh Rao

CURRENT

10000

304

Priya Singh

ACCOUNT_ID

CUSTOMER_NAME

ACCOUNT_TYPE

BALANCE

SAVINGS

8080

BEGIN

```
EXECUTE IMMEDIATE 'DROP TABLE employees';
```

```
EXCEPTION WHEN OTHERS THEN NULL;
```

```
END;
```

```
/
```

```
CREATE TABLE employees (
```

```
    employee_id NUMBER PRIMARY KEY,
```

```
    employee_name VARCHAR2(100),
```

```
    department VARCHAR2(50),
```

```
    salary NUMBER(12,2)
```

```
);
```

```
INSERT INTO employees VALUES (401, 'Karthik Iyer', 'SALES', 50000);
```

```
INSERT INTO employees VALUES (402, 'Meera Nair', 'SALES', 55000);
```

```
INSERT INTO employees VALUES (403, 'Arjun Reddy', 'IT', 70000);
```

```
INSERT INTO employees VALUES (404, 'Divya Menon', 'SALES', 48000);
```

```
INSERT INTO employees VALUES (405, 'Sanjay Gupta', 'MARKETING', 60000);
```

```
COMMIT;
```

```
-- ---- THE PROCEDURE ----
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
```

```
    p_department IN VARCHAR2,
```

```
    p_bonus_percentage IN NUMBER
```

```
) AS
```

```
    v_rows_updated NUMBER;
```

```
BEGIN
```

```
    IF p_bonus_percentage IS NULL OR p_bonus_percentage <= 0 THEN
```

```

        RAISE_APPLICATION_ERROR(-20001, 'Bonus percentage must be a positive
number');

    END IF;

v_rows_updated := 0;

FOR emp IN (
    SELECT employee_id, employee_name, salary
    FROM employees
    WHERE department = p_department
) LOOP
    UPDATE employees
    SET salary = salary + (salary * p_bonus_percentage / 100)
    WHERE employee_id = emp.employee_id;

    v_rows_updated := v_rows_updated + SQL%ROWCOUNT;

    DBMS_OUTPUT.PUT_LINE('Bonus applied to ' || emp.employee_name ||
        ' — Old salary: ' || emp.salary ||
        ', New salary: ' || ROUND(emp.salary * (1 + p_bonus_percentage / 100),
2));

END LOOP;

IF v_rows_updated = 0 THEN
    DBMS_OUTPUT.PUT_LINE('No employees found in department: ' || p_department);
END IF;

COMMIT;

EXCEPTION

```

```
WHEN OTHERS THEN

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('Error updating bonus: ' || SQLERRM);

    RAISE;

END;

/

-- ---- TEST CALLS ----

SET SERVEROUTPUT ON;

-- Give SALES department a 10% bonus
EXEC UpdateEmployeeBonus('SALES', 10);

-- Give IT department a 5% bonus
EXEC UpdateEmployeeBonus('IT', 5);

-- Department with no employees — should now correctly print "No employees found"
EXEC UpdateEmployeeBonus('HR', 10);

-- Verify
SELECT * FROM employees ORDER BY employee_id;
```

Bonus applied to Karthik Iyer – Old salary: 50000, New salary: 55000
 Bonus applied to Meera Nair – Old salary: 55000, New salary: 60500
 Bonus applied to Divya Menon – Old salary: 48000, New salary: 52800
 Bonus applied to Arjun Reddy – Old salary: 70000, New salary: 73500

Meera Nair

Meera Nair	
SALES	60500
EMPLOYEE_ID	
EMPLOYEE_NAME	
DEPARTMENT	SALARY
403	
Arjun Reddy	
IT	73500
404	
Divya Menon	
EMPLOYEE_ID	
EMPLOYEE_NAME	
DEPARTMENT	SALARY
SALES	52000
405	
Sanjay Gupta	
MARKETING	60000

BEGIN

EXECUTE IMMEDIATE 'DROP TABLE accounts';

EXCEPTION WHEN OTHERS THEN NULL;

END;

/

CREATE TABLE accounts (

account_id NUMBER PRIMARY KEY,

customer_name VARCHAR2(100),

balance NUMBER(12,2)

);

INSERT INTO accounts VALUES (501, 'Ravi Kumar', 50000);

INSERT INTO accounts VALUES (502, 'Anita Sharma', 3000);

INSERT INTO accounts VALUES (503, 'Suresh Rao', 15000);

COMMIT;

-- ---- THE PROCEDURE ----

CREATE OR REPLACE PROCEDURE TransferFunds (

 p_from_account IN NUMBER,

 p_to_account IN NUMBER,

 p_amount IN NUMBER

) AS

 v_from_balance NUMBER;

 v_to_exists NUMBER;

BEGIN

 IF p_amount IS NULL OR p_amount <= 0 THEN

 RAISE_APPLICATION_ERROR(-20003, 'Transfer amount must be a positive number');

 END IF;

 -- Check destination account exists

 SELECT COUNT(*) INTO v_to_exists FROM accounts WHERE account_id =
p_to_account;

 IF v_to_exists = 0 THEN

 RAISE_APPLICATION_ERROR(-20002, 'Destination account does not exist');

 END IF;

 -- Lock and read the source account balance

 SELECT balance INTO v_from_balance

 FROM accounts

 WHERE account_id = p_from_account

 FOR UPDATE;

```

IF v_from_balance < p_amount THEN

    RAISE_APPLICATION_ERROR(-20004, 'Insufficient balance in account ' ||
p_from_account ||

        ' (Available: ' || v_from_balance || ', Requested: ' || p_amount || ')');

END IF;


UPDATE accounts SET balance = balance - p_amount WHERE account_id =
p_from_account;

UPDATE accounts SET balance = balance + p_amount WHERE account_id =
p_to_account;


COMMIT;


DBMS_OUTPUT.PUT_LINE('Transferred ' || p_amount || ' from account ' ||
        p_from_account || ' to account ' || p_to_account);


EXCEPTION

WHEN NO_DATA_FOUND THEN

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('Error: Source account ' || p_from_account || ' does not
exist');

WHEN OTHERS THEN

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

/


-- ---- TEST CALLS ----

SET SERVEROUTPUT ON;

```

-- 1) Successful transfer

EXEC TransferFunds(501, 502, 10000);

-- 2) Failed transfer: insufficient balance (account 502 now has 13000, requesting 50000)

EXEC TransferFunds(502, 503, 50000);

-- 3) Failed transfer: destination account doesn't exist

EXEC TransferFunds(501, 999, 100);

-- Verify final balances

SELECT * FROM accounts ORDER BY account_id;

Output:

Transferred 10000 from account 501 to account 502
Error: ORA-20004: Insufficient balance in account 502 (Available: 13000,
Requested: 50000)
Error: ORA-20002: Destination account does not exist

```
ACCOUNT_ID
-----
CUSTOMER_NAME
-----
BALANCE
-----
501
Ravi Kumar
40000

502
Anita Sharma
13000
```

```
ACCOUNT_ID
-----
CUSTOMER_NAME
-----
BALANCE
-----
503
Suresh Rao
15000
```

```
ACCOUNT_ID
-----
CUSTOMER_NAME
-----
BALANCE
-----
501
Ravi Kumar
40000

502
Anita Sharma
13000
```

```
ACCOUNT_ID
-----
CUSTOMER_NAME
-----
BALANCE
-----
503
Suresh Rao
15000
```